



PROGETTO MACHINE LEARNING



INDICE

Il progetto ha previsto la creazione di un albero decisionale sfruttando i dati contenuti in un *dizionario*, sfruttando le colonne di interesse per creare un modello predittivo.

Lo svolgimento del progetto è riassumibile in 6 punti:

- Caricamento e Analisi preliminare dei dati.
- Preparazione dei dati.
- Analisi delle relazioni tra le variabili.
- Ottimizzazione del modello.
- Valutazione del modello finale.
- Conclusioni sulle analisi.



CARICAMENTO E ANALISI PRELIMINARE DEI DATI

Nella fase iniziale del progetto, sono state importate le principali librerie necessarie per la manipolazione dei dati e la costruzione del modello, come **pandas**, **numpy**, **matplotlib**, **seaborn** e gli strumenti di machine learning da **scikit-learn**.

Successivamente, è stato caricato il dataset *Wine*, disponibile tra i dataset di esempio forniti da quest'ultima.

Dopo il caricamento del dataset, inizialmente in formato dizionario, è stato trasformato in un **DataFrame** di *pandas*, per facilitarne l'analisi e la gestione.

Per una prima esplorazione visiva, è stato visualizzato un campione di righe iniziali tramite **print(df.head().to_string())**. Utilizzo il metodo **to_string()** per visualizzare tutte le colonne del *DataFrame* in modo leggibile e lineare, evitando i ritorni a capo automatici che *pandas* applica quando ci sono molte colonne.

Attraverso **df.info()**, è stato possibile ottenere una panoramica della struttura del dataset, comprendente il numero di righe e colonne, i tipi di dati presenti e la presenza o l'assenza di valori nulli.

Sono stati inoltre effettuati controlli di qualità sui dati:

- **df.duplicated().sum()** ha confermato l'assenza di righe duplicate
- **df.isnull().sum()** ha evidenziato che non sono presenti valori mancanti

Questa prima fase ha confermato che il dataset era già ben strutturato, completo e pronto per l'analisi esplorativa.

PREPARAZIONE DEI DATI

In questa fase, il dataset è stato suddiviso in un set di addestramento e uno di test utilizzando la funzione **train_test_split()**, applicando la stratificazione rispetto alla variabile **target**. Questo ha garantito che la distribuzione delle classi rimanesse proporzionata in entrambi i sottoinsiemi.

L'analisi preliminare delle classi ha evidenziato uno sbilanciamento di circa il 13%, ritenuto comunque gestibile senza interventi correttivi immediati.

Da questo punto in poi, tutte le analisi esplorative sono state effettuate esclusivamente sul **training set**, mantenendo il **test set** "invisibile" fino alla fase finale di validazione, per evitare qualsiasi forma di data leakage.

L'analisi descrittiva è iniziata con il metodo **describe()** sul set di training, utile per osservare statistiche come media, deviazione standard, minimi e massimi per ciascuna variabile. Successivamente è stata costruita una matrice di correlazione con **X_train.corr()**, e visualizzata mediante una **heatmap** tramite la libreria *seaborn*, che ha rivelato la presenza di alcune coppie di variabili moderatamente e fortemente correlate, suggerendo potenziale **multicollinearità**.

Per esplorare meglio la distribuzione delle variabili numeriche rispetto al **target**, è stata utilizzata una rappresentazione **boxplot tra feature e classi**, che ha permesso di individuare alcune classi, in base alla variabile, affette da **outlier**.

Infine, per preparare correttamente i dati ai modelli basati sulla distanza (come KNN) o alla regolarizzazione (come la regressione logistica), i dati sono stati **standardizzati** utilizzando **StandardScaler()**. Tale trasformazione ha centrato ogni feature sulla media 0 e le ha scalate in base alla deviazione standard, rendendo le variabili comparabili tra loro.

ANALISI DELLE RELAZIONI TRA VARIABILI

Per valutare l'effetto della standardizzazione e verificare eventuali criticità nei dati, è stato eseguito uno **spot check** comparativo su diversi modelli sia con i dati originali che con quelli standardizzati. I modelli testati includono:

- **Logistic Regression**
- **Random Forest**
- **K-Nearest Neighbors (KNN)**
- **Linear Regression**

I risultati hanno mostrato che modelli come **KNN** e **Logistic Regression** migliorano sensibilmente in presenza di dati standardizzati. Ad esempio, la precisione media del KNN è passata da circa **0.68** (con dati non scalati) a oltre **0.95** con dati scalati, dimostrando chiaramente quanto lo scaling possa impattare modelli basati sulla distanza.

Successivamente, è stata analizzata la **multicollinearità tra le variabili** tramite il calcolo del **VIF (Variance Inflation Factor)**. Nessuna feature ha mostrato valori di VIF superiori a 10, segnalando un'assenza di multicollinearità dannosa e suggerendo che tutte le feature forniscono informazioni utili e non ridondanti.

Per approfondire ulteriormente l'importanza delle variabili, sono stati applicati due metodi di selezione delle feature:

- **Mutual Information** (`mutual_info_classif()`)
- **Chi-quadro** (`chi2_contingency()`)

Entrambi hanno confermato l'importanza informativa di tutte le feature disponibili, suggerendo che nessuna poteva essere scartata.

ANALISI DELLE RELAZIONI TRA VARIABILI

Tuttavia, considerata la decisione di usare **Random Forest** come modello finale, si è deciso di sfruttare l'attributo di **feature importance** nativo del modello per identificare le variabili più rilevanti. Sono risultate particolarmente influenti (con importanza > 0.05) le seguenti feature:

- *color_intensity*
- *flavanoids*
- *proline*
- *alcohol*
- *hue*
- *od280/od315_of_diluted_wines*

Utilizzando solo queste variabili, è stato ripetuto lo *spot check*: in alcuni modelli, le performance sono **migliorate**, in particolare per la **Logistic Regression**, che ha raggiunto un'accuratezza media di circa **0.986**, e per **KNN**, che ha superato **0.979**.

Tuttavia, il modello **Random Forest** è rimasto **stabile** sia con il set completo di feature che con il set selezionato. Questo conferma che **Random Forest** è in grado di gestire efficacemente tutte le variabili, senza subire penalizzazioni a causa della presenza di feature multiple.

Pertanto, è stato deciso di **utilizzare tutte le feature** nel modello finale, sfruttando le sua capacità di gestire automaticamente la rilevanza e le interazioni tra le variabili.

Questo passaggio ha quindi dimostrato che una selezione mirata delle feature può non solo semplificare il modello, ma anche migliorarne le prestazioni, rendendo il sistema più interpretabile ed efficiente.

OTTIMIZZAZIONE DEL MODELLO

Per ottimizzare le prestazioni del modello, è stata effettuata una ricerca dei migliori *iperparametri* utilizzando il metodo **RandomizedSearchCV**. Questo è stato applicato sia sul set di dati scalato completo che su quello ridotto alle sole feature più importanti selezionate tramite la **feature importance** di Random Forest.

Il processo di ricerca ha coinvolto 40 combinazioni di parametri per ciascuna delle due versioni del set di dati. I parametri esplorati includevano vari valori per il numero di **alberi** (`n_estimators`), la **profondità massima** degli alberi (`max_depth`), il numero minimo di campioni per suddividere un nodo (`min_samples_split`), il numero minimo di campioni per foglia (`min_samples_leaf`), la scelta del **criterio** di suddivisione (Gini o Entropy), la **gestione del bilanciamento delle classi** e l'uso di **bootstrap**.

I risultati della ricerca hanno mostrato che l'accuratezza media della validazione incrociata del modello sui dati con tutte le feature è risultata pari a **0.99**. Il miglior modello è stato identificato con i seguenti *iperparametri*:

- `{n_estimators: 200, max_depth: 40, min_samples_split: 2, min_samples_leaf: 2, max_features: 'log2', bootstrap: False, class_weight: 'balanced', criterion: 'gini'}`

Il modello risultante ha fornito un **classification report** eccellente, con un punteggio di f1-score delle classi 0, 1, 2, rispettivamente di **1.00, 0.99, 0.99**.

Utilizzando il set con solo le feature selezionate, l'accuratezza media è rimasta molto alta, raggiungendo **0.98**, con una leggera riduzione delle prestazioni rispetto all'uso di tutte le feature. In questo caso, il miglior set di parametri includeva:

- `{n_estimators: 250, max_depth: 50, min_samples_split: 10, min_samples_leaf: 1, max_features: 'log2', bootstrap: True, class_weight: 'balanced', criterion: 'gini'}`

Il modello risultante ha fornito un **classification report** quasi alla pari dell'altro, con un punteggio di f1-score delle classi 0, 1, 2, rispettivamente di **0.99, 0.97, 0.97**.

Anche se le performance sono rimaste molto elevate, il set utilizzando tutte le feature è lievemente migliore.

VALUTAZIONE DEL MODELLO FINALE

Infine, è stato eseguito un test finale utilizzando il miglior modello (ovvero quello con i parametri ottimizzati sui dati scalati completi) sul test set tenuto da parte. Il modello ha ottenuto un'accuratezza di 1.0, con una matrice di confusione che ha confermato la perfezione delle previsioni, senza alcun errore nella classificazione delle classi 0, 1 e 2.

- Risultati sul Test Set:
 - Accuracy: 1.0
- Classification Report:
 - Precisione, Recall e F1-Score di 1.00 per tutte le classi.
- Confusion Matrix:
 - Nessun errore nella previsione per tutte le classi.

Questi risultati indicano che il modello ottimizzato ha raggiunto prestazioni eccezionali e può essere considerato molto robusto per questo e per i futuri test di classificazione.

CONCLUSIONI SULLE ANALISI

Effetto dello Scaling: I risultati degli spot check hanno mostrato che l'accuratezza del modello **K-Nearest Neighbors (KNN)** è migliorata notevolmente dopo l'applicazione dello scaling dei dati. Questo conferma che modelli basati sulla distanza, come il KNN, beneficiano molto dello scaling. Al contrario, il modello **Random Forest**, che non è sensibile alla scala dei dati, ha mostrato prestazioni simili sia sui dati scalati che non scalati, giustificando l'uso di tutte le feature senza perdere performance.

Selezione delle Feature: L'analisi dell'importanza delle feature, tramite **feature importance** di Random Forest, ha evidenziato che alcune variabili, come **color_intensity**, **flavanoids**, **proline**, **alcohol**, **hue**, e **od280/od315_of_diluted_wines**, sono effettivamente rilevanti per la classificazione. Tuttavia, queste feature non sono risultate così determinanti da giustificare la scelta di usarle come uniche feature nel modello **Random Forest**, il quale, come visto, ha ottenuto ottime performance anche utilizzando tutte le feature disponibili.

Ottimizzazione con RandomizedSearchCV: La ricerca dei parametri ottimali tramite **RandomizedSearchCV** ha prodotto un miglioramento significativo delle performance del modello sui dati scalati completi. L'accuratezza media in validazione incrociata è salita fino a **0.99**, indicando una capacità predittiva molto elevata. L'ottimizzazione ha consentito al modello **Random Forest** di ottenere risultati eccellenti senza mostrare segnali di overfitting.

Conferma sul Test Set: Il miglior modello, ottenuto utilizzando tutte le feature e i parametri ottimizzati, ha ottenuto un'accuratezza perfetta sul test set, confermando la sua robustezza. La confusion matrix ha mostrato che tutte le classi sono state classificate correttamente, senza alcun errore.

Concludendo, l'approccio basato su **Random Forest** ha dato risultati eccellenti, con un'accuratezza perfetta sia durante la fase di validazione che nel test finale, confermando l'importanza di un processo di tuning mirato per ottenere prestazioni elevate in problemi di classificazione complessi.

CONSIDERAZIONI FINALI

In conclusione, questo progetto è finalizzato a fare pratica con un modello ad albero decisionale di classificazione, sfruttando i dati contenuti in un *dizionario* per prevedere un output multiclasse tra 0 e 2 dato un input di diverse feature, è stato interessante creare e capire le potenzialità di questo tipo di modello.



LINK PROGETTO

All'interno di questo Drive è disponibile il notebook .ipynb:

[progetto ml - Google Drive](#)



This work is licensed under a Creative Commons Attribution 4.0 International License.

